

AI in Software Development

The New Reality for Developers



Why This Topic?

AI currently dominates the software industry

Uncertainty among many developers and students



https://miro.com/app/board/uXjVGSZiNoQ=?share_link_id=954402751059

Agenda

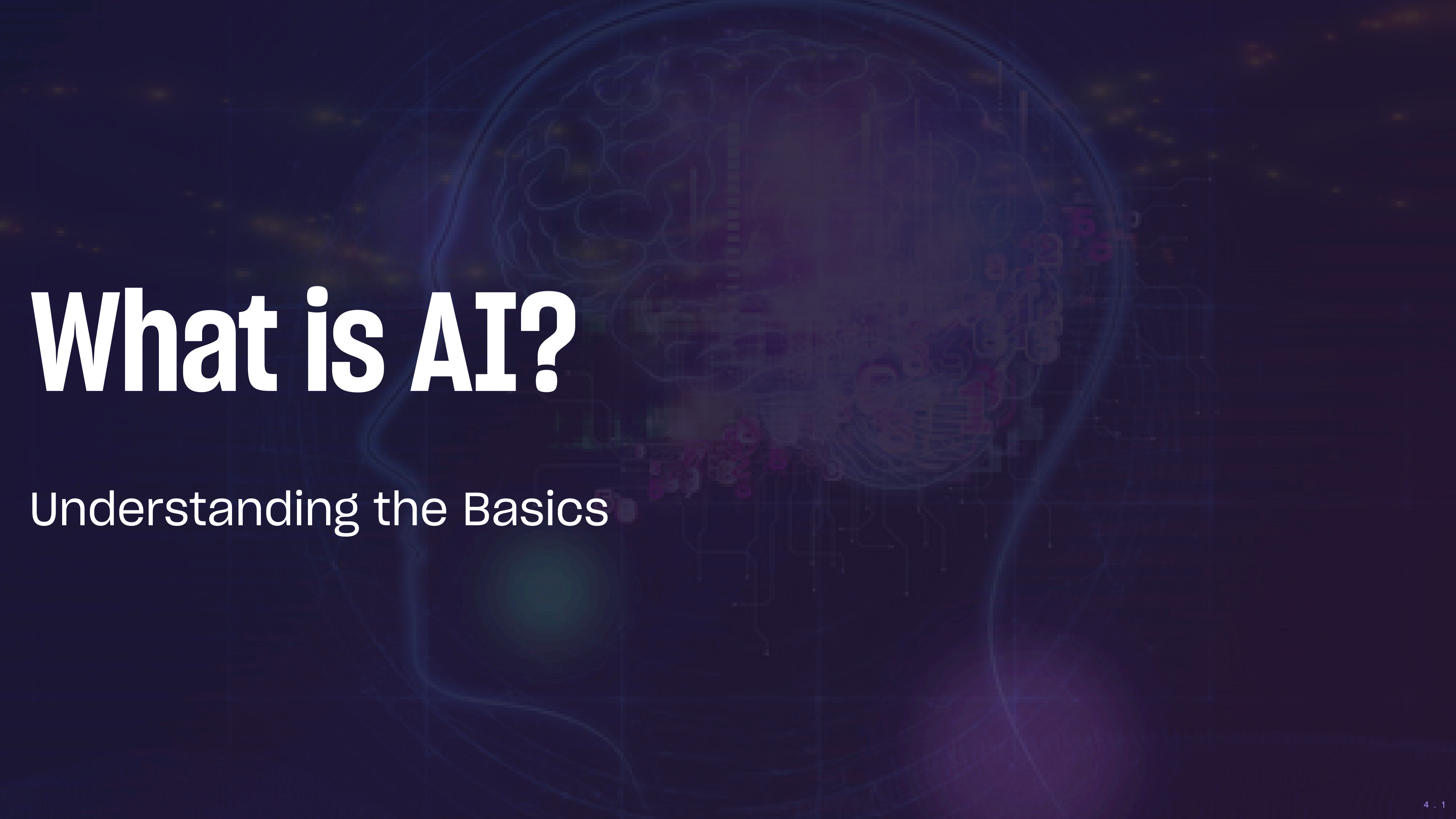
What is AI?

Terminology

Practical Example

Impact on Software Development

The Developer's Role in the Future



What is AI?

Understanding the Basics

AI is Not a New Phenomenon

Artificial Intelligence has existed since the 1950s

But: Breakthrough in recent years through:

- Massive computing power
- Large amounts of data
- New algorithms (Deep Learning, Transformer)

Types of AI

Narrow AI

Specialized in **one task**

Examples:

- Image recognition
- Language translation
- Playing chess

This is what we have today

General AI

Can do **any task** like a human

Not yet **achieved**

Subject of science fiction

Terminology

Important Concepts

Large Language Models (LLMs)

What is that?

Transformer models (Neural Networks), trained on huge amounts of text

Capabilities:

- Generate text based on instructions (Prompts)
- Generate code
- Create images, audio and videos

Examples: GPT-4o, Claude Sonnet 4.5, Gemini 2.5 Flash

AI Agents?

What is that?

Self contained systems that complete tasks **autonomously**

Characteristics:

- Understands goals and acts accordingly
- Executes actions autonomously
- Uses tools
- Learns independently

Agents vs. Chatbots

Chatbot

 Reactive

Responds to requests

Needs clear instructions

One exchange

Agent

 Proactive

Works independently

Plans multiple steps

Uses tools

Iterates until goal is reached

Agent Example: Code Review

Task: "Review my Pull Request"

Agent works autonomously:

1. Reads code changes
2. Performs static analysis
3. Searches for security vulnerabilities
4. Checks tests
5. Creates structured feedback
6. Suggests improvements

Prompting

What is that?

- Instruction given to the AI

Bad Prompt:

Write me a function

Good Prompt:

Write a TypeScript function that takes an array of numbers and returns the sum. Use functional programming. Add TypeScript types and JSDoc comment.

Prompt Engineering Principles

Provide Context

- Role, situation, goal

Be Specific

- Format, structure, constraints

Show Examples

- Few-shot learning

Refine Iteratively

- Feedback loop, Chain of Thought

Context Windows

What is that?

Maximum "memory span" of an LLM -> Short Term Memory

Previously: approx. 4,000 tokens (approx. 3,000 words)

Today: 200,000+ tokens

Implications:

- AI can now understand **entire projects**
- Longer sessions with multiple steps possible

Model Context Protocol

What is that?

- Protocol to connect LLMs with external data
- Access to external sources like APIs, files, tools
- Developed by Anthropic

Examples

- Github MCP Server
- Context7 MCP Server
- Playwright MCP Server

Skills

Specialized instruction sets that an AI agent **automatically recognizes and executes** based on context

They encapsulate complex, multi-step processes as reusable workflows — comparable to functions in code

This makes them particularly suited for **orchestrating standardized workflows**

Examples:

- **story-to-impl-plan** – Create implementation plan from user story
- **epic-to-stories** – Break epic into user stories

The AI Iceberg for Developers

Chat

e.g. ChatGPT

- Simple queries
- Prompting important
- Context important

CoPilot Systems

e.g. Github CoPilot

- Often integrated into the IDE
- Context is organized by CoPilot
- Inline Suggestions
- Guidelines important

Multi Agents

e.g. opencode

- Often integrated into the terminal
- Context Recompacting
- Specialized Subagents
- Integrated Permission System

Autonomous Agents

e.g. Claude Code

- Subagents with dedicated Context Windows
- PRDs or Spec KIT helpful
- Often deployed on separate servers



Practical Example





Impact on Development

What changes concretely?

What AI Can Already Do

- ✓ Generate boilerplate code
- ✓ Write tests
- ✓ Code refactoring
- ✓ Find and fix bugs
- ✓ Create documentation
- ✓ Code reviews
- ✓ Implement complex features

What AI Can't Do (Yet)

- ✗ Complex architecture decisions
- ✗ Stakeholder communication
- ✗ Understand business logic without context
- ✗ Creative problem solving with unclear requirements
- ✗ Long-term strategic planning

But: The boundary is shifting **quickly**

The Changed Workflow

Previously

Idea → Design → Write code
→ Test → Review → Deploy

Each step manual

Slow, but controlled

With AI

Idea → AI generates →
Review & Adjust → Test
(automated) → Deploy

Faster, but:

New skills required

Quality assurance more
important

Productivity Increase

Studies show:

- 40-50% faster development
- 60% more tasks completed
- Especially for junior developers

Source: GitHub Copilot Study 2023, McKinsey Developer Survey 2024

But:

Code quality must be monitored

New Risks

Security:

- Generated code may contain vulnerabilities

License Issues:

- Training on public code (GPL, etc.)

Hallucinations:

- AI sometimes invents APIs/functions

Over-Reliance:

- Unlearning fundamentals

Best Practices

✓ DO:

- AI as co-pilot, not as autopilot
- Always review and understand generated code
- Write tests (also AI-generated)
- Describe context well

✗ DON'T:

- Blind copy-paste
- Credentials in prompts
- Share confidential data
- Give up developer identity



The Future of Developers

Should I still pursue a degree?

The Central Question

Will AI replace software developers?

Short answer: No (Opinionated)

Long answer: The role is fundamentally changing

What Remains Important?

Problem Solving & Critical Thinking

AI provides solutions → You decide if they are good

Architecture & Design

Systems thinking remains human

Domain Knowledge

Understanding business context

Communication

With people, stakeholders, teams

The New Developer Role

Traditional

Code Writer

Syntax knowledge

Frameworks by heart

Debugging

Implementation

With AI

Solution Architect

Prompt Engineering

Systems thinking

Code Review

Orchestration

Quality Assurance

Will There Be Fewer Jobs?

Realistic Scenario:

Jobs change, not disappear

Why:

- More features = more demand
- New use cases possible
- AI itself must be developed
- Quality assurance becomes more important

But: Requirements increase

Should I Still Become a Developer?

YES!

But with different focuses:

- Fundamentals **more intensive** than ever
- Understand **why** something works
- Architecture and design more important
- Communication and context essential
- Master AI as a **tool**

Skills for the AI Era

Must-have:

1. **Solid fundamentals** (Algorithms, data structures)
2. **System Design** (Architectural thinking)
3. **Prompt Engineering** (Communicating with AI)
4. **Code Review** (Assessing quality)
5. **Testing** (Assurance)
6. **Domain Knowledge** (Understanding context)

How Do I Prepare?

Now:

- Use AI tools regularly
- Learn fundamentals **deeply**
- Understand what AI does well / poorly
- Experiment with Agents
- Stay curious

Long-term:

- Specialize (AI, Security, DevOps)
- Develop soft skills
- Understand business side

The AI Developer 2026+

Is simultaneously:

- Architect (Design decisions)
- Orchestrator (Coordinating AI tools)
- Quality Guardian (Review & Testing)
- Problem Solver (Creative solutions)
- Communicator (People & Machines)

No longer: Pure code writer

Open questions

1. How do you look into the future regarding your role / industry?
2. What are your experiences with AI tools so far?
3. Where do you see opportunities in this transformation?
And how are you preparing for it today?